

COMP5347 Individual Contribution Reflection

Gaoxuan Liu

1. Subsystem Responsibility and Implementation

My primary contribution to this project was the implementation and refinement of the leaderboard subsystem. This feature was responsible for presenting quiz performance results to users after quiz attempts had been submitted. Within the overall quiz application, the leaderboard served as a result-summary interface that connected stored quiz attempt data with a user-facing ranking page.

The leaderboard subsystem followed the MERN stack structure already used in the project. The frontend was built with React, while the backend used Express.js, MongoDB, and Mongoose. My work focused on connecting the leaderboard page to the backend data flow and refining the way leaderboard information was displayed to users.

The leaderboard feature supported the following functions:

- Displaying quiz result rankings through a dedicated leaderboard page.
- Connecting the React frontend leaderboard component to the backend leaderboard API.
- Presenting the main ranking information, including rank, username, score, and total questions.
- Removing unnecessary completion-time display from the leaderboard table.
- Handling the no-data condition through a clear empty-state message.
- Refining the leaderboard user interface after integration with the rest of the application.

A key part of my contribution was ensuring that the leaderboard page remained simple and focused. The team decided that the leaderboard should mainly show ranking-related information rather than extra metadata such as the exact time a quiz was completed. Based on this decision, I removed the completed-time column and kept the table focused on rank, username, score, and total questions.

I also improved the frontend presentation of the leaderboard page. This included adding clearer layout styling, improving the empty-state display, and making the page look more consistent with the rest of the quiz application. When no quiz attempts were available in the database, the page displayed a structured message explaining that scores would appear after users completed quizzes. This helped avoid a blank or confusing page state.

After later integration and styling changes, I identified and fixed a duplicated empty-state issue. The leaderboard page was showing the “No scores yet” message twice: once inside the styled leaderboard card and once again as plain text below the card. I corrected this by removing the unnecessary duplicate rendering logic and keeping only the styled empty-state display.

2. Major Technical Challenge and Resolution

One major technical challenge was understanding the relationship between the frontend leaderboard page, the backend API, and the local MongoDB database. During local testing, the leaderboard page loaded correctly but did not display any ranking data. I checked the backend API response and confirmed that it returned a successful response with an empty data array. This showed that the feature was not necessarily broken; instead, my local MongoDB database did not contain any quiz attempt records at that time.

This issue helped me understand the difference between an implementation bug and an empty database state. Since a fresh local database may not contain quiz attempts, the leaderboard needed to handle the no-data condition properly. I therefore focused on making the empty-state display clear and user-friendly rather than treating the empty leaderboard as an error.

Another challenge involved local development configuration. During testing, the frontend sometimes ran on localhost:3001 instead of localhost:3000 because port 3000 was already occupied. This caused CORS errors because the backend configuration allowed requests from localhost:3000, while the browser request came from

localhost:3001. I resolved this by stopping the existing Node processes and restarting the frontend on the correct port. This helped me better understand how browser origin checks work and why the protocol, host, and port all matter in frontend-backend communication.

A further challenge was avoiding overlap with other team members' work. Other parts of the system, including authentication, quiz logic, admin question management, and global styling, were being developed at the same time. To reduce the risk of merge conflicts, I kept my changes focused on leaderboard-related files and avoided modifying unrelated files such as login, register, quiz, admin, package-lock, or environment files. This made my contributions easier to review and safer to integrate.

3. Testing and Verification

I tested my contribution by running the full local development environment, including the backend server, frontend React application, and MongoDB database. I verified that the backend connected to MongoDB successfully and that the leaderboard endpoint could be accessed through the browser.

For frontend testing, I opened the leaderboard route and checked the page behaviour under the no-data condition. Since my local database did not contain quiz attempt records, the leaderboard correctly showed the empty-state message. After removing the completed-time column, I checked that the leaderboard table structure only contained the intended ranking fields. After fixing the duplicated empty-state issue, I refreshed the page and confirmed that the "No scores yet" message appeared only once inside the styled leaderboard card.

The main testing activities included:

- Running the backend server and confirming MongoDB connection.
- Running the frontend React application on the expected local port.
- Checking the leaderboard page route in the browser.
- Checking the backend leaderboard API response.
- Confirming that the empty-state display worked correctly when no scores were available.
- Confirming that the completed-time column was no longer displayed.
- Confirming that the duplicated empty-state message was removed.

This testing was important because leaderboard pages often need to work correctly both when data exists and when no data has been created yet. A clear empty state improves the user experience and makes the application appear more complete.

4. Git Commit History Analysis

The commits below summarise my direct code contributions to the leaderboard feature. I have only included commits that directly relate to my own work rather than merge commits or styling commits made by other team members.

Commit Hash	Explanation
84994a9	Refined the leaderboard frontend interface by removing the completed-time column and improving the leaderboard display structure. This made the page focus on the key ranking information: rank, username, score, and total questions.
c83b2d4	Fixed the duplicated leaderboard empty-state message after UI integration. The page now displays the styled “No scores yet” message only once when no quiz attempts are available.

These commits reflect the later refinement and bug-fixing stage of my leaderboard contribution. The work helped improve the clarity, consistency, and usability of the final leaderboard page.

5. Reflection on Design Decisions and Group Collaboration

A key design decision in my contribution was keeping the leaderboard page focused on the most useful ranking information. The original display included completion time, but this was not essential for the main purpose of the leaderboard. Removing the time column made the table cleaner and easier to read. The final display focused on rank, username, score, and total questions, which better matched the purpose of comparing quiz performance.

Another important design decision was improving the empty-state behaviour. A leaderboard page may have no data when the database is new or when no users have completed quizzes. Instead of showing a plain message or an empty table, the page needed to clearly explain why no results were shown. The styled empty-state message made this condition more understandable for users and made the interface appear more polished.

This contribution also improved my understanding of how a MERN-style application works across multiple layers. The leaderboard feature required frontend React rendering, backend API access, and MongoDB data availability. Through testing, I learned that a page showing no data does not always mean the code is incorrect. It can also be caused by an empty local database or missing test records. This made me more careful when diagnosing issues, because checking the API response and database state is just as important as checking the frontend component.

From a collaboration perspective, I learned the importance of keeping changes small and focused in a group project. Since other team members were working on authentication, quiz mechanics, admin functionality, and styling, it was important for me to avoid unnecessary changes outside my assigned area. I checked changed files before committing and avoided committing local environment files or unrelated dependency changes. This helped reduce the chance of conflicts and made my work easier for the team to review.

Overall, my contribution strengthened the result-viewing side of the quiz application. The leaderboard feature provided users with a clearer way to view quiz performance, while the later UI refinements improved the page’s readability and consistency. This work also helped me gain practical experience with frontend-backend integration, local development testing, Git workflow, and collaborative web application development.

6. Appendix

Figure 1: Leaderboard Data Flow Diagram

User -> React Frontend: Open leaderboard page
React Frontend -> Express API: GET /api/leaderboard
Express API -> MongoDB: Retrieve quiz attempt or score records
MongoDB -> Express API: Return leaderboard data
Express API -> React Frontend: Return JSON response
React Frontend -> User: Display ranking table or empty-state message

Figure 2: Leaderboard Empty-State Handling

User opens leaderboard page
-> React requests leaderboard API
-> API returns leaderboard data?
-> Yes: render leaderboard table
-> No: render styled empty-state message
-> No duplicate plain message displayed